PPS Technical Deep Dive - Part 3: Implementation Reference

ErthaLoka Planetary Performance System *Complete Parameter & Code Reference Guide*

Table of Contents

1. Complete Metric Parameters
2. Normalization Function Selection
3. Ecosystem-Adaptive Weights
4. Data Quality Score (DQS) Parameters
5. ESV Calculation Parameters
6. Grading Scale Reference
7. Dataset Catalog
8. Code Snippets Reference
9. File Structure Reference
10. TypeScript Type Definitions
11. Python Model Reference
12. Sources & References

---

1. Complete Metric Parameters

Category A: Atmospheric Health

Metric	v_min	v_max	v_mid/v_opt	Norm Type	Weight	Criticality
aod	0.0	1.5	-	`inverse_linear`	0.40	important
aqi	0	300	-	`inverse_linear`	0.40	important
uv_index	0	11	5.5 (k=0.5)	`sigmoid`	0.10	auxiliary
visibility	1	50	-	`linear`	0.10	auxiliary
cloud_fraction	0.0	1.0	-	`inverse_linear`	0.00	auxiliary

Notes:

- AOD (Aerosol Optical Depth): 0 = pristine air, >1 = very hazy/polluted
- AQI uses US EPA scale: 0-50 Good, 51-100 Moderate, 101-150 Unhealthy for Sensitive Groups
- UV Index uses WHO scale with sigmoid for moderate optimality

Category B: Biodiversity

Metric	v_min	v_max	v_mid/k	Norm Type	Weight	Criticality
ndvi	-0.1	0.9	-	`linear`	0.35	**critical**
evi	0.0	1.0	-	`linear`	0.35	important
lai	0.0	7.0	3.5 (k=0.5)	`sigmoid`	0.15	supporting
fpar	0.0	0.95	-	`linear`	0.15	supporting
land_cover	10	100	-	`linear`	0.00	auxiliary

Notes:

- NDVI: <0 water, 0-0.2 bare soil, 0.2-0.4 sparse vegetation, >0.6 dense vegetation

- EVI superior to NDVI in high-biomass regions and for canopy corrections
- LAI uses sigmoid for diminishing returns at high leaf density
- Land cover class is used for ecosystem detection, not scoring

Category C: Climate/Carbon

Metric	v_min	v_max	v_mid/k	Norm Type	Weight	Criticality
tree_cover	0	100	-	`linear`	0.35	**critical**
biomass	0	400	200 (k=0.5)	`sigmoid`	0.35	important
canopy_height	0	40	20 (k=0.5)	`sigmoid`	0.15	important
forest_loss	0	10	-	`inverse_linear`	0.15	supporting
carbon_stock	0	200	100 (k=0.5)	`sigmoid`	0.00	important

Notes:

- Tree cover from Hansen Global Forest Change (2000 baseline)
- Biomass measured in Mg/ha (megagrams per hectare)
- Carbon stock derived from biomass using factor of 0.5 (50% of biomass is carbon)
- Forest loss percentage since 2000 - lower is better

Category D: Decrease in Land & Water Degradation

Metric	v_min	v_max	v_opt/sigma	Norm Type	Weight	Criticality
soil_moisture	0.05	0.45	0.30 (sigma=0.12)	`gaussian`	0.35	**critical**
drought_index	-2.0	2.0	-	`centered`	0.35	important
lst	15	45	25 (sigma=8)	`gaussian`	0.15	supporting
evaporative_stress	-1.0	1.0	-	`centered`	0.15	supporting
water_occurrence	0	100	-	`linear`	0.00	auxiliary

Notes:

- Soil moisture uses Gaussian - optimal around 30% volumetric water content
- Drought index uses centered normalization - zero indicates no drought
- LST (Land Surface Temperature) optimal around 25°C for most ecosystems
- Evaporative stress index: negative = water stress, positive = energy limited

Category E: Ecosystem Services

Metric	v_min	v_max	v_mid/k	Norm Type	Weight	Criticality
human_modification	0.0	1.0	-	`inverse_linear`	0.50	**critical**
population	0	5000	500 (k=0.3)	`inverse_sigmoid`	0.17	supporting
nightlights	0	63	20 (k=0.3)	`inverse_sigmoid`	0.17	supporting
distance_to_water	0	50	-	`inverse_linear`	0.16	auxiliary
elevation	0	5000	-	`linear`	0.00	auxiliary

Notes:

- Human modification index: 0 = pristine wilderness, 1 = fully modified (urban core)

- Population density affects ecosystem integrity
- Nightlights proxy for urbanization intensity
- Distance to water measured in km - closer water sources generally beneficial

---

2. Normalization Function Selection

Decision Tree for Normalization Selection

```
START
  │
  ├─ Is higher always better? ──YES──> linear
  │
  ├─ Is lower always better? ──YES──> inverse_linear
  │
  ├─ Is there an optimal range (not at extremes)?
  │    │
  │    ├─ Is the optimal a single point? ──YES──> gaussian
  │    │
  │    └─ Is optimal around zero? ──YES──> centered
  │
  └─ Is there diminishing returns at high values?
       │
       ├─ Higher is better with saturation ──> sigmoid
       │
       └─ Lower is better with saturation ──> inverse_sigmoid
```

Normalization Function Assignments

Function	Parameters	Metrics Using
`linear`	`v_min`, `v_max`	ndvi, evi, fpar, tree_cover, visibility, land_cover, water_occurrence, elevation
`inverse_linear`	`v_min`, `v_max`	aod, aqi, cloud_fraction, forest_loss, human_modification, distance_to_water
`sigmoid`	`v_min`, `v_max`, `v_mid`, `k`	uv_index, lai, biomass, canopy_height, carbon_stock
`inverse_sigmoid`	`v_min`, `v_max`, `v_mid`, `k`	population, nightlights
`gaussian`	`v_min`, `v_max`, `v_opt`, `sigma`	lst, soil_moisture
`centered`	`v_min`, `v_max`	drought_index, evaporative_stress

---

3. Ecosystem-Adaptive Weights

Weight Distribution by Ecosystem Type

Ecosystem	A (Atm)	B (Bio)	C (Carbon)	D (Degrad)	E (Eco)	Priority Focus
tropical_forest	0.10	0.25	**0.35**	0.15	0.15	Carbon storage, biodiversity
mangrove	0.10	0.20	**0.30**	**0.25**	0.15	Carbon, water stress
grassland_savanna	0.15	**0.30**	0.15	**0.25**	0.15	Biodiversity, degradation
wetland	0.10	0.25	0.20	**0.30**	0.15	Water/soil conditions
agricultural	0.20	0.25	0.10	0.25	0.20	Balanced, air quality

urban_green	**0.30**	0.25	0.10	0.15	0.20	Atmospheric quality
default	0.20	0.20	0.20	0.20	0.20	Equal weighting

Land Cover to Ecosystem Mapping

WorldCover Class	Value	Ecosystem Type
Tree cover	10	tropical_forest
Shrubland	20	grassland_savanna
Grassland	30	grassland_savanna
Cropland	40	agricultural
Built-up	50	urban_green
Bare/sparse vegetation	60	default
Snow and ice	70	default
Permanent water	80	wetland
Herbaceous wetland	90	wetland
Mangroves	95	mangrove
Moss and lichen	100	default

---

4. Data Quality Score (DQS) Parameters

Criticality Weights

Criticality Level	Weight	Critical Metrics
critical	1.0	ndvi, tree_cover, soil_moisture, human_modification
important	0.7	aod, aqi, evi, biomass, canopy_height, drought_index, carbon_stock
supporting	0.4	lai, fpar, forest_loss, lst, population, nightlights, evaporative_stress
auxiliary	0.2	uv_index, visibility, cloud_fraction, land_cover, water_occurrence, elevation, distance_to_water

DQS Thresholds

Threshold	Value	Meaning
`category_minimum`	40	Minimum DQS for valid category score
`overall_minimum`	50	Minimum DQS for valid overall PPS
`investment_grade`	70	Threshold for investment-grade data quality
`high_confidence`	85	Threshold for high confidence results

DQS Calculation Formula

```
DQS = [Σ(w_i × a_i) / Σw_i] × 100

Where:
- w_i = criticality weight for metric i
- a_i = 1 if metric available with acceptable quality, 0 otherwise
- Quality adjustment: good=1.0, moderate=0.8, poor=0.5, unavailable=0
```

---

5. ESV Calculation Parameters

Base ESV Values by Ecosystem Type

Ecosystem Type	Base ESV ($/ha/year)	Source
Wetland	$25,682	Costanza et al. (2014)
Mangrove	$9,990	de Groot et al. (2012)
Tropical Forest	$5,382	Costanza et al. (2014)
Urban Green	$3,212	Regional estimates
Grassland/Savanna	$2,871	Costanza et al. (2014)
Agricultural	$1,532	TEEB Database
Default/Mixed	$3,000	Global average

PHI-ESV Multiplier Constants

```
PHI_ESV_CONSTANTS = {
    "k": 0.6,        # Base sensitivity factor
    "alpha": 0.15    # Logarithmic acceleration factor
}
```

Multiplier Formula

```
M_PHI = [(PHI − 50) / 100] × k × [1 + α × ln(PHI / 50)]

Where:
- PHI = overall PPS score (0-100)
- k = 0.6 (base sensitivity)
- α = 0.15 (logarithmic acceleration)

Result range: -0.30 to +0.36
```

ESV Service Breakdown Multipliers

Service Category	Multiplier
Provisioning (food, water, materials)	0.20
Regulating (climate, water, pollination)	0.35
Cultural (recreation, aesthetic)	0.15
Supporting (nutrient cycling, soil)	0.30

Carbon Credit Price Range

Price Level	$/tonne CO2
Low (VCS/Gold Standard floor)	$15
Mid (Average voluntary market)	$25
High (Premium/Compliance markets)	$50

---

6. Grading Scale Reference

PPS Score to Grade Mapping

Grade	Score Range	Label	Color
AAA	86-100	Excellent	`#22c55e` (green)
AA	72-85	Very Good	`#4ade80` (light green)
A	58-71	Good	`#84cc16` (lime)
BBB	44-57	Above Average	`#facc15` (yellow)
BB	30-43	Average	`#f59e0b` (amber)
B	16-29	Below Average	`#f97316` (orange)
CCC	0-15	Poor	`#ef4444` (red)

Grade Implementation

```
// src/types/phi.ts
export const getGradeFromScore = (score: number | null | undefined): ScoreInterpretation => {
  if (score === null || score === undefined) return 'Unavailable';
  if (score >= 86) return 'AAA';
  if (score >= 72) return 'AA';
  if (score >= 58) return 'A';
  if (score >= 44) return 'BBB';
  if (score >= 30) return 'BB';
  if (score >= 16) return 'B';
  return 'CCC';
};
```

---

7. Dataset Catalog

Complete GEE Dataset Reference

Dataset ID	Collection	Resolution	Temporal	Metrics
`MODIS/061/MCD19A2_GRANULES`	MODIS AOD	1km	Daily	aod
`MODIS/061/MOD08_M3`	MODIS Atmosphere	111km	Monthly	aqi, uv, cloud
`COPERNICUS/S2_SR_HARMONIZED`	Sentinel-2	10m	5-day	RGB, NIR
`MODIS/061/MOD13A2`	MODIS Vegetation	1km	16-day	ndvi, evi

`MODIS/061/MOD15A2H`	MODIS LAI/FPAR	500m	8-day	lai, fpar
`ESA/WorldCover/v200`	ESA WorldCover	10m	Annual	land_cover
`UMD/hansen/global_forest_change_2023_v1_11`	Hansen GFC	30m	Annual	tree_cover, loss
`LARSE/GEDI/GEDI04_A_002_MONTHLY`	GEDI Biomass	1km	Monthly	biomass
`users/nlang/ETH_GlobalCanopyHeight_2020_10m_v1`	ETH Canopy	10m	Static	canopy_height
`MODIS/061/MOD11A2`	MODIS LST	1km	8-day	lst_day, lst_night
`NASA/SMAP/SPL4SMGP/007`	SMAP Soil	11km	3-hourly	soil_moisture
`ECMWF/ERA5_LAND/MONTHLY_AGGR`	ERA5-Land	11km	Monthly	soil, temp, precip
`JRC/GSW1_4/GlobalSurfaceWater`	JRC Water	30m	Static	water_occurrence
`MODIS/061/MOD16A2`	MODIS ET	500m	8-day	et, pet
`WorldPop/GP/100m/pop_age_sex_cons_unadj`	WorldPop	100m	Annual	population
`NOAA/VIIRS/DNB/MONTHLY_V1/VCMSLCFG`	VIIRS DNB	500m	Monthly	nightlights
`CSP/HM/GlobalHumanModification`	CSP HM	1km	Static	human_modification
`USGS/SRTMGL1_003`	SRTM	30m	Static	elevation

Scale Factors and Units

Dataset	Band	Scale Factor	Offset	Unit
MODIS AOD	Optical_Depth_047	0.001	0	dimensionless
MODIS NDVI	NDVI	0.0001	0	index
MODIS EVI	EVI	0.0001	0	index
MODIS LAI	Lai_500m	0.1	0	m^2/m^2
MODIS LST	LST_Day_1km	0.02	-273.15	°C
GEDI Biomass	agbd	1.0	0	Mg/ha
VIIRS DNB	avg_rad	1.0	0	$nW/cm^2/sr$

---

8. Code Snippets Reference

Normalization Functions (Python)

```python
# utils/normalization.py

def linear_normalize(value: float, v_min: float, v_max: float) -> float:
    """Linear normalization: higher is better."""
    if value is None:
        return None
    clamped = max(v_min, min(v_max, value))
    return ((clamped - v_min) / (v_max - v_min)) * 100

def inverse_linear_normalize(value: float, v_min: float, v_max: float) -> float:
```

```python
    """Inverse linear: lower is better."""
    if value is None:
        return None
    clamped = max(v_min, min(v_max, value))
    return (1 - (clamped - v_min) / (v_max - v_min)) * 100

def sigmoid_normalize(value: float, v_min: float, v_max: float, v_mid: float, k: float) -> float:
    """Sigmoid: diminishing returns at high values."""
    if value is None:
        return None
    normalized = (value - v_min) / (v_max - v_min)
    mid_norm = (v_mid - v_min) / (v_max - v_min)
    sigmoid_value = 1 / (1 + math.exp(-k * (normalized - mid_norm) * 10))
    return sigmoid_value * 100

def gaussian_normalize(value: float, v_min: float, v_max: float, v_opt: float, sigma: float) -> float:
    """Gaussian: optimal at specific value."""
    if value is None:
        return None
    deviation = (value - v_opt) / sigma
    return math.exp(-0.5 * deviation ** 2) * 100

def centered_normalize(value: float, v_min: float, v_max: float) -> float:
    """Centered: zero is optimal."""
    if value is None:
        return None
    abs_max = max(abs(v_min), abs(v_max))
    return (1 - abs(value) / abs_max) * 100
```

ESV Calculation (TypeScript)

```typescript
// utils/esvCalculation.ts

export const calculatePHIMultiplier = (phi: number): number => {
  if (phi === null || phi === undefined || phi <= 0) return 0;

  const k = 0.6;
  const alpha = 0.15;
  const phiDeviation = (phi - 50) / 100;
  const logFactor = phi > 0 ? Math.log(phi / 50) : 0;

  return phiDeviation * k * (1 + alpha * logFactor);
};

export const calculateESV = (
  area: number,
  ecosystemType: EcosystemType,
  phi: number
): ESVResult => {
  const baseValue = BASE_ESV_VALUES[ecosystemType] || 3000;
  const multiplier = calculatePHIMultiplier(phi);
  const adjustedValue = baseValue * (1 + multiplier);

  return {
    baseESV: baseValue,
```

```
    phiMultiplier: multiplier,
    adjustedESV: adjustedValue,
    totalAnnual: adjustedValue * area,
    services: calculateServiceBreakdown(adjustedValue, area)
  };
};
```

DQS Calculation (Python)

```
# utils/quality.py

CRITICALITY_WEIGHTS = {
    "critical": 1.0,
    "important": 0.7,
    "supporting": 0.4,
    "auxiliary": 0.2
}

def calculate_dqs(
    metrics_availability: Dict[str, bool],
    quality_flags: Dict[str, str]
) -> float:
    """
    Calculate Data Quality Score with criticality weighting.
    """
    weighted_sum = 0.0
    weight_total = 0.0

    for metric_name, is_available in metrics_availability.items():
        params = PHI_METRIC_PARAMS.get(metric_name, {})
        criticality = params.get("criticality", "auxiliary")
        weight = CRITICALITY_WEIGHTS.get(criticality, 0.2)

        if is_available:
            quality = quality_flags.get(metric_name, "good")
            quality_factor = {"good": 1.0, "moderate": 0.8, "poor": 0.5}.get(quality, 1.0)
            weighted_sum += weight * quality_factor

        weight_total += weight

    return (weighted_sum / weight_total) * 100 if weight_total > 0 else 0
```

Pillar Score Calculation (Python)

```
# utils/scoring.py

def calculate_pillar_score(pillar_id: str, metrics: Dict[str, Any]) -> Optional[int]:
    """Calculate weighted pillar score from normalized metrics."""

    weighted_sum = 0.0
    weight_total = 0.0

    for metric_name, metric_data in metrics.items():
        if metric_name not in PHI_METRIC_PARAMS:
```

```
            continue

        params = PHI_METRIC_PARAMS[metric_name]
        if params.get("category") != pillar_id:
            continue

        value = metric_data.get("value") if isinstance(metric_data, dict) else metric_data
        if value is None:
            continue

        # Normalize the value
        normalized = normalize_metric(metric_name, value, params)
        if normalized is None:
            continue

        weight = params.get("weight", 0)
        weighted_sum += normalized * weight
        weight_total += weight

    if weight_total == 0:
        return None

    return round(weighted_sum / weight_total)
```

---

9. File Structure Reference

Complete Project Structure

```
main-1/
├── src/                            # Frontend (React + TypeScript)
│   ├── components/
│   │   ├── ui/                     # Radix + shadcn components
│   │   ├── charts/
│   │   │   ├── ChartConfig.ts      # Chart.js configuration
│   │   │   ├── PillarRadar.tsx     # Radar chart component
│   │   │   └── index.ts
│   │   ├── ESVBreakdown.tsx        # ESV visualization
│   │   └── MapComponent.tsx        # Leaflet map
│   ├── services/
│   │   ├── phiApi.ts               # PPS API client (L:1-594)
│   │   ├── metricSupplementation.ts # External data merge (L:1-227)
│   │   ├── sensorApi.ts            # IoT sensor service
│   │   ├── apiMiddleware.ts        # Response transformation
│   │   ├── adminService.ts         # Admin operations
│   │   └── instituteAuth.ts        # Organization auth
│   ├── types/
│   │   ├── phi.ts                  # PPS type definitions (L:1-336)
│   │   ├── weather.ts              # Weather data types
│   │   ├── institute.ts            # Organization types
│   │   └── admin.ts                # Admin types
│   ├── utils/
│   │   ├── esvCalculation.ts       # ESV computation (L:1-282)
│   │   ├── esvProjection.ts        # 5-year projections (L:1-282)
│   │   ├── impactScore/
│   │   │   ├── calculator.ts       # Impact scoring
```

```
│   │   │   ├── types.ts
│   │   │   └── index.ts
│   │   └── impactAssessment.ts
│   ├── hooks/
│   │   ├── useSensorData.ts          # Real-time sensor hook
│   │   └── useEnvironmentalScore.ts  # Score calculation hook
│   ├── auth/
│   │   ├── firebase.ts               # Firebase configuration
│   │   ├── authService.ts            # Auth operations
│   │   └── index.ts
│   ├── db/
│   │   ├── firestore.ts              # Firestore client
│   │   ├── sensorService.ts          # Sensor data service
│   │   ├── types.ts
│   │   └── index.ts
│   ├── pages/
│   │   ├── Dashboard.tsx
│   │   ├── GetTheReport.tsx          # Main PPS report page
│   │   ├── LandAnalysis.tsx          # Polygon analysis
│   │   └── ...
│   └── lib/
│       └── utils.ts                  # Utility functions
│
├── Hello-/                           # Backend (Python + FastAPI)
│   ├── app.py                        # FastAPI main (L:1-242)
│   ├── requirements.txt              # Python dependencies
│   ├── planetary_health_query/
│   │   ├── __init__.py
│   │   ├── core/
│   │   │   ├── config.py             # ALL parameters (L:1-786)
│   │   │   ├── engine.py             # GEEQueryEngine (L:1-940)
│   │   │   └── authenticator.py      # GEE auth
│   │   ├── pillars/
│   │   │   ├── __init__.py
│   │   │   ├── base.py               # BasePillar class (L:1-321)
│   │   │   ├── atmospheric.py        # Pillar A
│   │   │   ├── biodiversity.py       # Pillar B
│   │   │   ├── carbon.py             # Pillar C
│   │   │   ├── degradation.py        # Pillar D
│   │   │   └── ecosystem.py          # Pillar E
│   │   └── utils/
│   │       ├── normalization.py      # 6 norm functions
│   │       ├── scoring.py            # Score calculation
│   │       └── quality.py            # DQS calculation
│   └── backend/
│       ├── requirements.txt
│       └── external_apis/
│           ├── open_meteo.py         # Weather integration
│           └── openaq.py             # Air quality API
│
├── New-f/                            # Documentation
│   ├── PPS_Technical_Deep_Dive_Part1_Formulas_Algorithms.md
│   ├── PPS_Technical_Deep_Dive_Part2_Architecture_Technology.md
│   └── PPS_Technical_Deep_Dive_Part3_Implementation_Reference.md
│
├── package.json                      # Frontend dependencies (L:1-105)
```

```
├── tsconfig.json                    # TypeScript config
├── vite.config.ts                   # Vite build config
├── tailwind.config.js               # TailwindCSS config
└── README.md
```

---

10. TypeScript Type Definitions

Core Types (src/types/phi.ts)

```typescript
// Request types
export interface PPAQueryRequest {
  latitude: number;
  longitude: number;
  mode?: 'simple' | 'comprehensive';
  temporal?: 'latest' | 'monthly' | 'annual';
  buffer_radius?: number;
  pillars?: string[];
}

// Metric data structure
export interface MetricData {
  value: number | null;
  unit?: string;
  description?: string;
  quality?: 'good' | 'moderate' | 'poor' | 'unavailable' | 'supplemented';
  source?: string;
  resolution?: string;
  error?: string;
  [key: string]: unknown;
}

// Pillar data structure
export interface PillarData {
  metrics: Record<string, MetricData>;
  pillar_id: string;
  pillar_name: string;
  pillar_color: string;
  score: number | null;
  data_date?: string;
  mode: string;
  query_time: string;
  error?: string;
}

// Score interpretation (AAA to CCC)
export type ScoreInterpretation =
  | 'AAA' | 'AA' | 'A'
  | 'BBB' | 'BB' | 'B'
  | 'CCC'
  | 'Unavailable';

// Ecosystem types
export type EcosystemType =
  | 'tropical_forest'
  | 'mangrove'
```

```
  | 'grassland_savanna'
  | 'wetland'
  | 'agricultural'
  | 'urban_green'
  | 'default';

// Summary structure
export interface PPASummary {
  overall_score: number;
  overall_interpretation?: ScoreInterpretation;
  pillar_scores: Record<string, number>;
  ecosystem_type?: EcosystemType;
  ecosystem_weights?: Record<string, number>;
  data_quality_score?: number;
  data_completeness: number;
  dqs_recommendation?: string;
  missing_critical_metrics?: string[];
  quality_flags: string[];
  esv_multiplier?: number;
  methodology?: string;
}

// Full API response
export interface PPAResponse {
  query: QueryMetadata;
  pillars: Record<string, PillarData>;
  summary: PPASummary;
  time_series: TimeSeriesInfo;
}
```

Pillar Configuration

```
export const PILLAR_CONFIGS: Record<string, PillarConfig> = {
  A: {
    id: 'A',
    name: 'Atmospheric',
    fullName: 'Atmospheric Health',
    color: '#3498db',
    icon: 'Wind',
    description: 'Air quality, aerosol optical depth, UV index'
  },
  B: {
    id: 'B',
    name: 'Biodiversity',
    fullName: 'Biodiversity',
    color: '#27ae60',
    icon: 'Leaf',
    description: 'Vegetation indices, land cover classification'
  },
  C: {
    id: 'C',
    name: 'Climate',
    fullName: 'Climate',
    color: '#8e44ad',
    icon: 'TreePine',
```

```
    description: 'Forest cover, biomass, carbon sequestration'
  },
  D: {
    id: 'D',
    name: 'DLWD',
    fullName: 'Decrease in Land & Water Degradation',
    color: '#e74c3c',
    icon: 'Thermometer',
    description: 'Land surface temperature, soil moisture, drought'
  },
  E: {
    id: 'E',
    name: 'Ecosystem',
    fullName: 'Ecosystem Services',
    color: '#f39c12',
    icon: 'Globe',
    description: 'Population density, human modification, elevation'
  }
};
```

---

11. Python Model Reference

FastAPI Request Models

```
# app.py

class PHIQueryRequest(BaseModel):
    """Request model for PHI query."""
    latitude: float = Field(..., ge=-90, le=90, description="Latitude")
    longitude: float = Field(..., ge=-180, le=180, description="Longitude")
    mode: str = Field(default="comprehensive")
    temporal: str = Field(default="latest")
    buffer_radius: int = Field(default=500, ge=100, le=10000)
    pillars: Optional[List[str]] = None

class PolygonQueryRequest(BaseModel):
    """Request model for polygon query."""
    points: List[Dict[str, float]]  # [{"lat": x, "lng": y}, ...]
    mode: str = Field(default="comprehensive")
    include_scores: bool = Field(default=True)
    user_id: Optional[str] = None
    user_email: Optional[str] = None

class PHISummary(BaseModel):
    """Response model for summary."""
    overall_score: int
    pillar_scores: Dict[str, int]
    data_completeness: float
    quality_flags: List[str]
    ecosystem_type: Optional[str] = None
    data_quality_score: Optional[float] = None
    esv_multiplier: Optional[float] = None
```

Carbon Credit Response Model

```
class CarbonCreditsResponse(BaseModel):
    available: bool
    carbon_stock_mg_c_ha: Optional[float] = None
    total_carbon_mg: Optional[float] = None
    co2_equivalent_tonnes: Optional[float] = None
    verified_co2_tonnes: Optional[float] = None
    confidence_factor: Optional[float] = None
    estimated_value: Optional[Dict[str, float]] = None
    methodology: str = "IPCC Tier 1"
```

---

12. Sources & References

Scientific Literature

1. **Costanza, R., et al. (2014)**
   - "Changes in the global value of ecosystem services"
   - *Global Environmental Change*, 26, 152-158
   - Used for: Base ESV values
2. **de Groot, R., et al. (2012)**
   - "Global estimates of the value of ecosystems and their services"
   - *Ecosystem Services*, 1(1), 50-61
   - Used for: Ecosystem service valuation
3. **Hansen, M.C., et al. (2013)**
   - "High-Resolution Global Maps of 21st-Century Forest Cover Change"
   - *Science*, 342(6160), 850-853
   - Used for: Forest cover metrics
4. **IPCC (2019)**
   - "2019 Refinement to the 2006 IPCC Guidelines for National Greenhouse Gas Inventories"
   - Used for: Carbon calculation methodology

Satellite Data Sources

Source	Agency	Citation
MODIS	NASA	Vermote, E. (2015). MOD09A1 MODIS/Terra Surface Reflectance 8-Day L3 Global 500m SIN Grid V006.
Sentinel-2	ESA	Drusch, M., et al. (2012). Sentinel-2: ESA's Optical High-Resolution Mission.
GEDI	NASA	Dubayah, R., et al. (2020). The Global Ecosystem Dynamics Investigation.
SRTM	NASA/NGA	Farr, T.G., et al. (2007). The Shuttle Radar Topography Mission.

External APIs

API	Provider	Documentation
Open-Meteo	Open-Meteo	https://open-meteo.com/en/docs
OpenAQ	OpenAQ	https://docs.openaq.org/

Google Earth Engine	Google	https://developers.google.com/earth-engine
Firebase	Google	https://firebase.google.com/docs
Supabase	Supabase	https://supabase.com/docs

Standards & Frameworks

Standard	Organization	Use Case
US AQI	EPA	Air Quality Index scale
WorldCover	ESA	Land cover classification
TEEB	UNEP	Ecosystem valuation framework
Verra VCS	Verra	Carbon credit verification
Gold Standard	Gold Standard	Premium carbon credits

Quick Reference Cards

Normalization Quick Reference

```
LINEAR:           Score = (v - v_min) / (v_max - v_min) × 100
INVERSE_LINEAR:   Score = [1 - (v - v_min) / (v_max - v_min)] × 100
SIGMOID:          Score = 1 / [1 + e^(-k(x-mid)×10)] × 100
INVERSE_SIGMOID:  Score = 1 / [1 + e^(k(x-mid)×10)] × 100
GAUSSIAN:         Score = e^[-0.5((v-v_opt)/σ)²] × 100
CENTERED:         Score = [1 - |v|/|v_max|] × 100
```

Grade Quick Reference

```
AAA (86-100) = Excellent      | BBB (44-57) = Above Average
AA  (72-85)  = Very Good      | BB  (30-43) = Average
A   (58-71)  = Good           | B   (16-29) = Below Average
                              | CCC (0-15)  = Poor
```

ESV Quick Reference

```
Wetland:     $25,682/ha/yr    | Urban Green:  $3,212/ha/yr
Mangrove:    $9,990/ha/yr     | Grassland:    $2,871/ha/yr
Forest:      $5,382/ha/yr     | Agricultural: $1,532/ha/yr
```

DQS Quick Reference

```
Critical (1.0):   ndvi, tree_cover, soil_moisture, human_modification
Important (0.7):  aod, aqi, evi, biomass, drought_index
Supporting (0.4): lai, fpar, lst, population, nightlights
Auxiliary (0.2):  uv_index, cloud_fraction, elevation
```

Document Version: 3.0 Last Updated: December 2024 Part of PPS Technical Deep Dive Series

Complete Series:

- Part 1: Formulas & Algorithms
- Part 2: Architecture & Technology
- Part 3: Implementation Reference (this document)